System Analysis Document

Team Name
git force push

Domain 2
AI for Economic Empowerment & Decision Intelligence

Team Member	Role
Brayden Chong Jie Rui	Team Lead, Market Analyst Agent & System Orchestrator
Cheah Yi Chern	Operations Advisor Agent, Backend Integration
Chin Yiu Ern	Frontend Development & QA Testing
Goh Sheng Fung	Location Scouting Agent, Backend Integration
Ong Yean	Machine Learning Engineer, Inventory Planner Agent

1. Introduction

1.1 Purpose

This document explains the technical design and system architecture of **BijakSense**, an AI-powered decision co-pilot for SME merchants.

It focuses on:

- How the system is structured
- How data flows through the system
- How AI (GLM) is used for decision-making
- Key technical decisions behind the product

1.2 Background

SME merchants in Southeast Asia make critical business decisions daily without access to structured market intelligence.

BijakSense solves this by:

- Monitoring real-world signals
- Processing them using AI agents
- Converting them into clear, actionable business decisions

2. Target Stakeholders

Stakeholder	Role	Expectation
SME Merchant	Uses dashboard, map, and chat	Clear, fast decisions in plain language
Development Team	Builds system	Modular architecture, clear APIs
QA Engineer	Tests system	Defined outputs, edge case coverage
Judges	Evaluate solution	Real AI reasoning, not just UI

3. System Architecture

3.1 Overview

Type	Details
System	Web Application
Frontend	React + TypeScript
Backend	FastAPI

Architecture	Multi-agent AI system

BijakSense follows a **client-server architecture**:

- Frontend (React) → displays insights
- Backend (FastAPI) → handles logic & AI orchestration
- AI Layer (GLM) → generates decisions

3.2 Core Architecture Flow

World Signals → Relevance Filter → Orchestrator → Agents → GLM → Recommendations → Dashboard

Explanation (simple):

- External signals are collected
- Only relevant signals are selected
- Orchestrator sends them to the right agent
- Agents use GLM to generate decisions
- Results are shown to the merchant

3.3 Multi-Agent System

Component	Role
Orchestrator	Routes signals to correct agent
Inventory Planner	Handles supply & pricing risks

Market Analyst	Handles trends & competitors
Location Scout	Handles expansion decisions
Ops Advisor	Handles staffing & operations

Each agent:

- Receives structured input
- Calls GLM
- Returns structured JSON output

3.4 LLM Integration (GLM)

GLM is used as a **reasoning engine**, not just a chatbot.

Each request contains:

- Merchant context
- Signal data
- Agent role
- Instruction

Output:

- Structured recommendation (JSON)

If output fails:

- Retry once
- Fallback to cached data

4. Data Flow

4.1 High-Level Flow

1. External APIs provide signals (weather, trends, etc.)
2. Signals are filtered based on merchant relevance
3. Orchestrator routes signals to agents
4. Agents generate recommendations using GLM
5. Results are stored in PostgreSQL
6. Dashboard retrieves and displays results

4.2 Storage

Component	**Usage**
PostgreSQL	Stores signals, recommendations
Redis	Caching & agent state
Celery	Background signal processing

5. Technology Stack

Layer	Technology	Justification
Frontend	React + TypeScript + Tailwind CSS	Component-based architecture suits the multi-page dashboard; TypeScript enforces API contract compliance
Map	Leaflet.js	Lightweight, open-source, supports custom markers and animated polylines for signal arcs
Charts	Recharts	Declarative React-native charts for price trend sparklines on Inventory Planner page
Backend	Python + FastAPI	Async-native, fast to develop, native support for StreamingResponse for chat endpoint
Agent Orchestration	LangGraph	Manages agent state machines and routing logic; supports conditional branching for signal type routing
Task Queue	Celery + Redis broker	Handles asynchronous signal ingestion and processing without blocking the API
Primary LLM	Z.AI GLM	Required by domain problem statement; supports structured JSON output essential for A2A agent communication; strong multilingual reasoning for SEA context

Agent Protocol	A2A over HTTP	Each agent is a stateless FastAPI endpoint; Orchestrator calls agents as HTTP services
Tool Interface	MCP-compatible schemas	Each agent exposes a defined tool schema allowing Orchestrator to invoke agents as tools within a GLM tool-use call
Database	PostgreSQL + SQLAlchemy async	Relational structure suits merchant profiles, signal records, and recommendation history; async driver required for FastAPI
Cache / Session	Redis + aioredis	Agent status tracking, context window caching, signal deduplication with 2-hour TTL
Deployment	Docker Compose	Single-machine demo deployment; production-ready path to Kubernetes

6. Functional Scope

Minimum Viable Product (MVP)

	Feature	Description
	Intelligence Map	Shows global and local signals
	Dashboard	Displays prioritized insights
	AI Chat	Ask questions, get decisions
	Inventory Planner	Stock & pricing recommendations
	Market Analyst	Trends & competitor insights

7. Non-Functional Requirements

Quality	Requirement	Implementation
Performance	< 200ms dashboard load	Cached data only
Latency	< 5s AI response	Streaming response
Reliability	No demo failure	Fallback data
Scalability	Handle multiple users	Async + Redis
Cost	Control API usage	Caching + limits

8. Out of Scope (Future Work)

- Real-time production data feeds
- POS integration
- Payment systems
- Mobile app
- Automated ordering

9. Monitoring & Risk Handling

Priority Matrix

Priority	Issue	Action
P0	AI fails	Use cached data
P1	Slow response	Reduce context
P2	Data delay	Retry processing

Monitoring

- API response time
- AI error rate
- Cache hit rate
- Queue load

10. Assumptions

- Users have stable internet
- Demo data represents real-world scenarios
- GLM API is available
- Judges evaluate based on demo

11. External Dependencies

Tool	Purpose	Risk
GLM API	Core AI reasoning	High
Weather API	Climate signals	Medium
News API	Market signals	Medium
Redis	Caching	Low

12. Project Timeline

Day	Task
1	Planning
2–3	Backend + agents
4–5	Frontend
6–7	Integration

8	Testing
9	Demo prep
10	Submission

13. Team Contributions

Member	Role
Brayden Chong Jie Rui	Team Lead · Orchestrator · Market Analyst
Ong Yean	ML Engineer · Inventory Planner
Cheah Yi Chern	Ops Advisor
Goh Sheng Fung	Location Scout
Chin Yiu Ern	Frontend · QA

14. Key Improvements

- Multi-agent architecture (not single AI)
- GLM used for reasoning, not chat
- Real-world signals → real decisions
- Clean separation of components
- Fast, cached, reliable system